



Universidad Experimental Nacional de Guayana

Vicerrectorado Académico

Coordinación General de Pregrado

P.F.G: Ingeniería en Informática

Unidad Curricular: INGENIERÍA DE SOFTWARE II

Producto I

Profesor:

Marquez Felix

Estudiantes:

Alondra Moreno CI: 30.331.70

Samuel Cortez CI: 27.158.842

Carlos Hernández CI: 27.975.753

Eduardo Quintero CI: 32.004.965

Ciudad Guayana, Septiembre 2026

Indice

Introducción.....	2
Ejercicio 1: Traducción de Necesidades a Requerimientos y Actores.....	3
Identificación de Actores, Necesidades y Requerimientos de Software:.....	3
Reflexión: Validación mediante IA y Prototipado antes de programar.....	5
Ejercicio 2: Análisis de las 4 Dimensiones y Errores Clásicos.....	5
1. Desbalances en las 4 Dimensiones de Pressman:.....	6
2. Identificación de 3 Errores Clásicos:.....	7
3. Propuesta de Mitigación desde el Rol de Líder Servicial:.....	7
Ejercicio 3: PMBOK 6 (Procesos) vs. PMBOK 7 (Principios).....	8
Diferencia Central:.....	8
La Analogía del Viaje en Auto:.....	8
Cambio en la Gestión de Requisitos de Software:.....	8
2 Ejemplos Prácticos en Desarrollo de Software:.....	9
Ejercicio 4: Discusión sobre el Riesgo en la Ingeniería de Software Moderna.....	10
Anexo: Prompt Experto para Investigación PMBOK 7 vs. PMBOK 8.....	10
Conclusión.....	11

Introducción

La unidad introductoria establece los cimientos conceptuales para comprender la evolución de la disciplina, desde la crisis del software hasta la era ágil y DevOps, pasando por el cambio de paradigma de la gestión de proyectos hacia la gestión de productos orientada a la entrega continua de valor. En este sentido, los ejercicios aquí resueltos abordan de manera integral los tópicos fundamentales: la correcta identificación de actores y su traducción a requerimientos funcionales y no funcionales; el análisis sistémico de proyectos de software desde las cuatro dimensiones de Pressman (Personas, Proceso, Producto y Tecnología); la comparación crítica entre los enfoques del PMBOK 6 y PMBOK 7; y la reflexión sobre los riesgos modernos en ingeniería de software, particularmente aquel de construir soluciones técnicamente impecables que nadie utiliza.

Cada ejercicio ha sido desarrollado aplicando los estándares internacionales de referencia, tales como el SWEBOK V4, el PMBOK 7, la ISO/IEC/IEEE 29148 para ingeniería de requisitos, y la ISO 31000 para gestión de riesgos, con el propósito de ofrecer respuestas rigurosas y contextualizadas en entornos reales de desarrollo de software. Asimismo, se incorporan reflexiones sobre el uso de tecnologías emergentes como la inteligencia artificial y el prototipado rápido como herramientas de validación temprana, en línea con las prácticas modernas de la industria

Ejercicio 1: Traducción de Necesidades a Requerimientos y Actores

Contexto: Un hospital público necesita reducir el tiempo de espera de los pacientes en urgencias.

Identificación de Actores, Necesidades y Requerimientos de Software:

- **1. Paciente de Urgencias (Usuario final) - Necesidad:** Incertidumbre y ansiedad por desconocer su posición en la cola de atención y el tiempo de espera estimado según la gravedad de su caso.
- **Requerimiento del Software (Funcional):** El sistema debe proveer una interfaz (pantalla en sala de espera o aplicación móvil) que muestre en tiempo real el turno del paciente, su estado actual en el flujo de atención y el tiempo de espera estimado basado en el nivel de triaje asignado.
- **2. Enfermero/a de Triage (Operativo) - Necesidad:** Pérdida de tiempo y riesgo de errores al registrar el historial y los síntomas del paciente en papel, dificultando la clasificación rápida.
- **Requerimiento del Software (Funcional):** El sistema debe incluir un formulario digital intuitivo para el registro de síntomas que se autocompleta basándose en el historial clínico previo del paciente, permitiendo registrar la información en menos de 2 minutos y asignando automáticamente un nivel de prioridad.
- **3. Médico de Guardia (Operativo/Especialista) - Necesidad:** Sobresaturación de trabajo por una mala distribución de pacientes durante su turno, lo que disminuye la calidad de la atención y genera agotamiento.
- **Requerimiento del Software (Funcional):** El sistema debe contar con un módulo de asignación algorítmica que distribuya a los pacientes entre los médicos disponibles,

bloqueando la asignación automática cuando un médico alcance su límite máximo de capacidad operativa.

- **4. Director / Administrador del Hospital (Estratégico) - Necesidad:** Falta de visibilidad sobre los insumos médicos gastados en urgencias, lo que dificulta la contabilidad, el presupuesto y el reabastecimiento oportuno.
- **Requerimiento del Software (Funcional/No Funcional):** El sistema debe integrar los reportes de atención médica con el módulo de inventario, descontando automáticamente los insumos utilizados en cada paciente y generando alertas de stock mínimo para reabastecimiento.

Reflexión: Validación mediante IA y Prototipado antes de programar

El uso de herramientas de prototipado rápido (como Figma) nos permite construir interfaces visuales interactivas antes de escribir código. Esto facilita validar tempranamente con los médicos y enfermeros si el sistema es realmente intuitivo y se adapta a la dinámica de una emergencia real. De esta forma, fomentamos la colaboración continua entre el equipo de desarrollo y los usuarios finales (cumpliendo el Manifiesto Ágil), mitigando el riesgo de construir un software que nadie entienda o use.

Por su parte, la Inteligencia Artificial actúa como un validador de factibilidad técnica. Al entrenar un modelo con el historial médico y operativo del hospital, la IA puede identificar patrones para predecir con alta precisión los tiempos de espera de los pacientes y los picos de demanda. Esto nos confirma, antes de programar, que el requerimiento de 'estimar el tiempo de espera' es técnicamente viable y aportará valor real.

Ejercicio 2: Análisis de las 4 Dimensiones y Errores Clásicos

Contexto: Una startup decide desarrollar una app de delivery. El gerente técnico impone el uso de microservicios en Kubernetes (Tecnología), pero el equipo solo sabe programar en PHP monolítico (Personas). No definieron un proceso de gestión de cambios (Proceso) y el cliente pide agregar una función de 'pagos con criptomonedas' a mitad del desarrollo sin ajustar el presupuesto (Producto/Alcance).

1. Desbalances en las 4 Dimensiones de Pressman:

Dimensión	Estado en la Startup	Diagnóstico del Desbalance
Tecnología vs. Personas	Imposición de Kubernetes y Microservicios frente a un equipo capacitado en PHP Monolítico.	Brecha técnica severa. La falta de competencias del capital humano frente a una arquitectura distribuida compleja genera cuellos de botella, frustración y deuda técnica técnica/ operativa prematura.
Proceso	Ausencia total de un protocolo o marco para la gestión y control de cambios.	Falta de gobernanza. No existen canales para canalizar requerimientos emergentes ni evaluar el impacto técnico o

		financiero de las solicitudes del cliente.
Producto / Alcance	Inclusión arbitraria de "pagos con criptomonedas" en pleno desarrollo.	Vulneración de la línea base del producto (MVP). Se altera la propuesta de valor sin recalibrar el Triángulo de Hierro (Costo, Tiempo, Alcance).

2. Identificación de 3 Errores Clásicos:

- **1. Scope Creep (Alcance no controlado):** Adición continua e incontrolada de requerimientos (pagos con criptomonedas) sin ajustar tiempo ni presupuesto.
- **2. Optimismo Injustificado:** Subestimación de la complejidad al asumir que el equipo aprenderá sobre la marcha una tecnología avanzada (Kubernetes) sin impactar el plazo de entrega.
- **3. Falta de Liderazgo Servicial y Mala Comunicación:** Dirección dictatorial ('comando y control') que impone decisiones tecnológicas sin consultar la realidad del equipo, desconectando la gestión de las capacidades reales.

3. Propuesta de Mitigación desde el Rol de Líder Servicial:

- **Protección y Empoderamiento del Equipo:** Facilitar una reunión técnica con el equipo para sincerar capacidades. Pivotar la arquitectura hacia PHP monolítico (lo que el equipo

domina) para asegurar la entrega de un Producto Mínimo Viable (MVP) a tiempo y con calidad.

- **Control de Cambios y Negociación:** Implementar un proceso formal para evaluar solicitudes. Explicar al cliente el impacto en costos y tiempos antes de aceptar cambios.
- **Enfoque en Entrega de Valor:** Asesorar al cliente para postergar la integración de criptomonedas a una segunda fase. Explicar que para un MVP de delivery, las criptomonedas no resuelven la necesidad inmediata del usuario final y añaden volatilidad e incertidumbre innecesaria.

Ejercicio 3: PMBOK 6 (Procesos) vs. PMBOK 7 (Principios)

Diferencia Central:

El PMBOK 6 se centra en el cumplimiento estricto de procesos, fases y áreas de conocimiento para entregar un output prefijado (enfoque predictivo). Por el contrario, el PMBOK 7 adopta un enfoque holístico basado en 12 Principios orientados a la Entrega de Valor y Resultados (outcomes), permitiendo una mayor adaptabilidad metodológica (tailoring).

La Analogía del Viaje en Auto:

- **PMBOK 6 (Basado en Procesos):** Es como seguir un mapa impreso en papel con una ruta inflexible. Define horas exactas de salida, paradas y llegada. Si una carretera está cerrada, hay que tramitar un permiso formal para cambiar la ruta. El éxito se mide por haber seguido el plan inicial.
- **PMBOK 7 (Basado en Principios):** Es como utilizar un GPS con información de tráfico en tiempo real. El principio orientador es llegar al destino de forma segura y eficiente. Si hay un

bloqueo, se recalcula la ruta al instante sin burocracia, priorizando el objetivo final sobre la ruta original.

Cambio en la Gestión de Requisitos de Software:

Bajo el PMBOK 6, los requisitos se tratan como un contrato cerrado y congelado en un documento exhaustivo (SRS). Cualquier modificación exige atravesar comités pesados de control de cambios.

Bajo el PMBOK 7, los requisitos son hipótesis de valor evolutivas organizadas en un backlog priorizado. La pregunta constante no es '¿estaba esto en la especificación inicial?', sino '¿esta funcionalidad sigue aportando el mayor valor al negocio hoy?'.

2 Ejemplos Prácticos en Desarrollo de Software:

Ejemplo 1: Módulo de Autenticación y Seguridad

- **Enfoque PMBOK 6:** El requisito inicial exige login por lector de huella dactilar USB. A mitad del proyecto, la empresa cliente adquiere laptops con cámara facial. Bajo PMBOK 6, cambiar el requisito requiere congelar el desarrollo y gestionar un cambio formal; si se rechaza por cronograma, se entrega un módulo obsoleto.
- **Enfoque PMBOK 7:** Guiados por el principio de 'entrega de valor y adaptabilidad', el equipo detecta el cambio de contexto y reordena el backlog en el siguiente Sprint, reemplazando la huella USB por reconocimiento facial sin fricción burocrática.

Ejemplo 2: Reporte de Ventas en App Móvil

- **Enfoque PMBOK 6:** Se firmó un contrato para desarrollar un reporte en PDF de 50 columnas exportable en móviles. El equipo invierte semanas optimizando el código, pero en

la pantalla del teléfono la tabla es ilegible. Se cumplió el contrato (output), pero el producto no sirve.

- **Enfoque PMBOK 7:** Enfocados en resultados ('outcomes'), el equipo diseña un dashboard móvil con 4 tarjetas KPI clave y opción de envío de informe por correo. Tras probarlo con usuarios, se confirma que resuelve la necesidad real con menor esfuerzo técnico.

Ejercicio 4: Discusión sobre el Riesgo en la Ingeniería de Software Moderna

La afirmación de que 'el mayor riesgo no es que el código no compile, sino que el software se construya perfectamente pero nadie lo use' encapsula la evolución de la ingeniería de software contemporánea. Hoy en día, escribir código sintácticamente correcto es una expectativa básica. Un software con una arquitectura pulcra pero que no resuelve un problema real del usuario es un rotundo fracaso.

Esta paradoja se entiende analizando la Crisis del Software (1968), donde los proyectos fallaban no por incapacidad técnica de codificación, sino por deficiencias estructurales en la comunicación, el liderazgo y la gestión de requisitos. Si no se identifican adecuadamente a los actores clave (stakeholders) y sus necesidades latentes, se corre el riesgo de construir soluciones técnicamente perfectas para los problemas equivocados.

Asimismo, la Curva de Boehm demuestra que corregir un fallo de requerimientos en etapas tardías o post-lanzamiento cuesta hasta 100 veces más que detectarlo en la fase de elicitación. Para mitigar este riesgo, la ingeniería moderna exige priorizar la construcción del producto correcto mediante la colaboración continua, el prototipado y la entrega incremental de valor.

Anexo: Prompt Experto para Investigación PMBOK 7 vs. PMBOK 8

A continuación se adjunta el prompt diseñado bajo el rol de Senior Project Manager para realizar la comparativa académica requerida en la entrega bonus:

"Actúa como un Senior Project Manager y experto académico en metodologías de Ingeniería de Software. Necesito que realices un análisis comparativo riguroso y exhaustivo entre el PMBOK 7ma Edición (2021) y las actualizaciones recientes o proyectadas del PMBOK 8va Edición (2026).

Por favor, estructura tu respuesta de la siguiente manera:

- 1. Evolución Filosófica: ¿Cuál es el cambio de paradigma principal entre la versión 7 y la 8?*
- 2. Comparativa Técnica: Una tabla o lista detallada contrastando los dominios, principios o áreas de conocimiento que se añadieron, eliminaron o transformaron.*
- 3. Impacto en Ingeniería de Software: Un análisis crítico de las ventajas y desventajas que esta nueva edición supone específicamente para la gestión de proyectos de desarrollo de software y productos digitales.*

El tono debe ser estrictamente académico, objetivo y crítico. Utiliza terminología profesional de gestión de proyectos y desarrollo de software."

Conclusión

El desarrollo de los ejercicios de la Unidad 1 ha permitido comprender que la gestión de proyectos de software trasciende la aplicación de herramientas técnicas, exigiendo un enfoque integral que articule personas, procesos, producto y tecnología. La correcta identificación de actores y la traducción de sus necesidades en requerimientos verificables, fundamentada en estándares como la ISO/IEC/IEEE 29148 y el SWEBOK V4, constituye el pilar para construir software que realmente aporte valor.

El análisis de casos prácticos evidenció que errores clásicos como el *scope creep*, el optimismo injustificado y la imposición tecnológica desalineada con las capacidades del equipo siguen siendo causas recurrentes de fracaso, y que el liderazgo servicial se presenta como el enfoque más efectivo para mitigarlos. Asimismo, la transición del PMBOK 6 hacia el PMBOK 7 refleja una evolución necesaria desde el control de procesos hacia la entrega continua de valor, adaptándose a la complejidad de los entornos tecnológicos actuales.

Finalmente, se concluye que el mayor riesgo en la ingeniería de software moderna no es técnico, sino de negocio: construir soluciones perfectas que nadie utiliza. Por ello, la ingeniería de requisitos y la identificación temprana de actores se consolidan como disciplinas estratégicas para garantizar la sostenibilidad e impacto de los productos digitales.

Bibliografia

Boehm, B. W. (1981). *Software Engineering Economics*. Prentice Hall.

Brooks, F. P. (1975). *The Mythical Man-Month: Essays on Software Engineering*.
Addison-Wesley.

Greenleaf, R. K. (1977). *Servant Leadership: A Journey into the Nature of Legitimate Power and Greatness*. Paulist Press.

IEEE Computer Society. (2014). *Guide to the Software Engineering Body of Knowledge (SWEBOK V4)*. IEEE.